Agentic-SQL Taxonomy: A Survey of Autonomous and Interactive Text-to-SQL with LLMs

No Author Given

No Institute Given

Abstract. Text-to-SQL systems have transitioned from simple machine translation models to complex reasoning frameworks as database schemas grow in scale and ambiguity. Despite the impressive capabilities of Large Language Models, one-shot generation often fails to produce correct SQL in real-world scenarios. This survey introduces the Agentic-SQL Taxonomy, an autonomy-based classification that reevaluates existing methods through the lens of inference complexity. We categorize research into single-turn generation, iterative refinement, and multi-agent collaboration to highlight the shift toward interactive debugging and collective reasoning. We analyze how these sophisticated pipelines bridge the performance gap on challenging benchmarks. Current evaluations show that leading models can result in incorrect execution in nearly 40 % of cases when instructions are vague or incomplete. Our work identifies execution-guided feedback and modular agent architectures as the primary drivers of future progress in building robust and reliable database interfaces.

Keywords: Text-to-SQL · Large Language Models · Agentic-SQL Taxonomy.

1 Introduction

Natural language interfaces to databases have evolved from brittle, rule-based systems to sophisticated neural architectures that treat query generation as a machine translation task. Today, the integration of Large Language Models (LLMs) has pushed the boundaries of what is possible, enabling users to query data using complex, conversational language.

However, the "translation" perspective is reaching its limits. As we move from academic datasets to real-world applications like BIRD [14] and Spider 2.0 [24] , the limitations of one-shot generation become clear. Modern enterprise databases contain thousands of tables, ambiguous column names, and domain-specific logic that a model cannot fully grasp in a single forward pass. Current evidence suggests that even state-of-the-art models like ChatGPT struggle with synonym variations and incomplete instructions, often leading to execution failure rates as high as 40% in complex scenarios [14]. The recent Spider 2.0 benchmark reveals an even harsher reality, where task solving accuracy for top-tier systems remains around 21.3%.

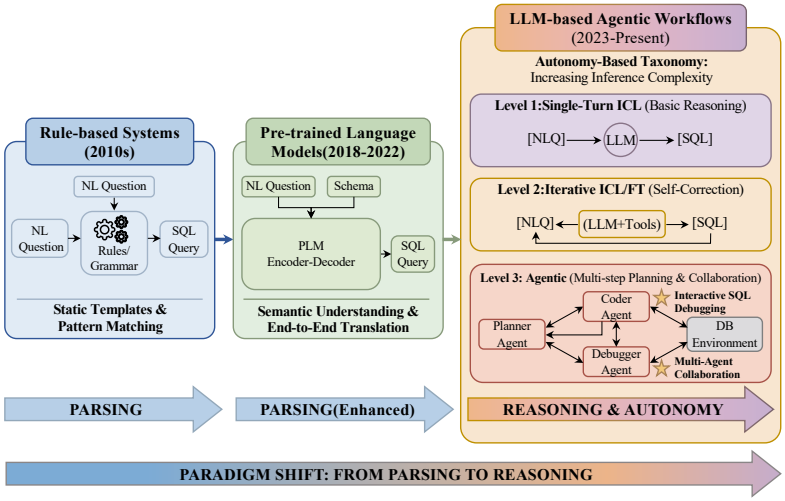

Fig. 1: Evolution of Text-to-SQL from static translation to agentic reasoning loops.

Traditional surveys in this field categorize research based on training paradigms, distinguishing between In-Context Learning (ICL) [3] and Supervised Fine-Tuning (SFT) [5]. While this distinction is technically accurate, it misses the most important shift occurring in the research community. The focus is moving away from how the model is trained and toward how it behaves during inference. High-performing systems like DIN-SQL [17] and CodeS [13] are increasingly employing multi-stage pipelines, self-correction loops, and interactive debugging to arrive at the correct answer. We argue that the level of autonomy in these reasoning loops is the primary driver of performance in the modern era.

This paper introduces the *Agentic-SQL Taxonomy*, a novel framework for classifying Text-to-SQL systems based on their inference complexity and autonomy. Instead of focusing solely on the model architecture, we examine the entire pipeline as an agentic workflow. We categorize existing work into three levels: Single-Turn Generation, Iterative Refinement, and Agentic Collaboration. This perspective allows us to analyze how components like execution feedback and multi-agent coordination solve problems that were previously intractable for monolithic models.

Our primary contributions are summarized below:

- We propose the Agentic-SQL Taxonomy, which reclassifies Text-to-SQL research based on three levels of inference autonomy and complexity.
- We provide a systematic review of the transition from translation-based models to interactive frameworks, highlighting the roles of self-correction and execution-guided refinement.
- We analyze the impact of specialized code models and prompt optimization strategies on real-world benchmarks like BIRD and Spider 2.0.

– We identify the fundamental trade-offs between reasoning depth and execution efficiency, outlining a roadmap for future research in private, efficient, and dialect-aware database agents.

2 Preliminaries

In this section, we formalize the Text-to-SQL task and describe the datasets and metrics that form the basis of current research.

2.1 Problem Formulation

The core objective of a Text-to-SQL task is to transform a natural language question into a structured query language (SQL) statement that can be executed against a specific database. We formally define the database schema as $S = \{T, C, R\}$, where T represents the set of tables, C denotes the columns within those tables, and R indicates the foreign key relationships that link them. Follow the BIRD benchmark [14], the system also receives external knowledge K to help disambiguate domain-specific terms.

When using LLMs for this task, the process is typically modeled as a conditional generation problem. Given an instruction I, a user question Q, and the schema context S, the LLM generates a predicted SQL query $\hat{Y}$. This can be expressed as:

$$\hat{Y} = \pi(I, Q, S, K|\theta) \tag{1}$$

Here, π represents the LLM parameterized by θ. In the ICL paradigm, θ remains frozen, and the model relies on the prompt to understand the mapping. For systems that employ fine-tuning, we optimize the model by minimizing the cross-entropy loss over a training dataset D:

$$\mathcal{L}_{SFT} = -\sum_{(P,Y)\in D}\sum_{t=1}^{L}\log Pr_{\pi}(y_t|P, y_{<t}), \tag{2}$$

P is the combined input prompt and y_t is the token generated at step t. While early models focused on a single-pass generation of $\hat{Y}$, the agentic frameworks we discuss in later sections treat this as an iterative process where $\hat{Y}$ is refined based on execution feedback.

2.2 Datasets: From Spider to BIRD

Text-to-SQL progress is tightly coupled with increasingly challenging benchmarks. Early datasets such as WikiSQL [10] established the task setup, but largely focused on simple queries and single-table schemas, limiting evaluation of compositional reasoning.

Spider [24] introduced a key shift toward cross-domain generalization: models must translate questions into SQL over unseen databases with multi-table

Table 1: High-level comparison of representative Text-to-SQL benchmarks.

Aspect	Spider	BIRD	Spider 2.0
Release	2018	2023	2024
Focus	Schema generalization	Realistic values & large schemas	Enterprise tasks & dialects
Task	Single-turn NL→SQL	NL→SQL (value-centric)	Workflow-style, multi-step
Schema scale	Medium (multi-table)	Large (wide schemas)	Very large (multi-schema)
Value realism	Moderate	High	High
Knowledge need	Limited	Important (entity linking)	Often required
SQL dialects	Mostly single	Mostly single	Multiple (e.g., warehouses)
Main challenges	Joins, nesting, unseen DBs	Value grounding, scale	Dialects, debugging, long horizon

schemas. It also provides difficulty levels (Easy/Medium/Hard/Extra Hard) based on joins and SQL compositionality (e.g., nested queries), making it a long-standing standard for structured generalization.

As performance on Spider became saturated, BIRD [14] moved evaluation closer to real deployments by emphasizing large schemas, realistic database values, and stronger reliance on external knowledge for entity linking. More recently, Spider 2.0 extends this trajectory to enterprise settings (e.g., complex SQL dialects and workflow-like tasks), where current state-of-the-art systems still struggle, highlighting a gap between academic benchmarks and production-grade database environments.

2.3 Evaluation Metrics

Evaluating the quality of generated SQL is difficult because a single question can often be answered by multiple valid SQL statements. Researchers use several key metrics to measure performance:

Exact Matching (EM): This metric compares the predicted SQL query $\hat{Y}$ directly against the ground truth query Y. While EM was common in early work, it is often too restrictive. It penalizes models that produce a correct but syntactically different query, such as using a different join order or alias [24].

Execution Accuracy (EX): This is the current gold standard for evaluation. EX measures whether the result set from executing $\hat{Y}$ matches the result set of the gold query Y. This metric captures the functional correctness of the code. However, it can occasionally yield false positives if a query is logically incorrect but happens to return the same result on a specific database instance.

Valid Efficiency Score (VES): Introduced alongside the BIRD benchmark [14], VES measures the computational efficiency of generated SQL. In large-scale databases, a query that takes minutes to run is less useful than one optimized for performance. VES rewards models that generate efficient execution plans, a key requirement for professional database administrators.

3 Paradigm I: In-Context Learning and Prompting

Modern Text-to-SQL research has split into two primary implementation tracks: In-Context Learning (ICL) and Fine-Tuning (FT) [15]. In this section, we intro-

duce our Agentic-SQL Taxonomy and describe the methodologies that define the ICL paradigm. Unlike early semantic parsing that required extensive architectural changes, ICL uses frozen Large Language Models (LLMs) by providing a small number of demonstrations or specific instructions within the input prompt.

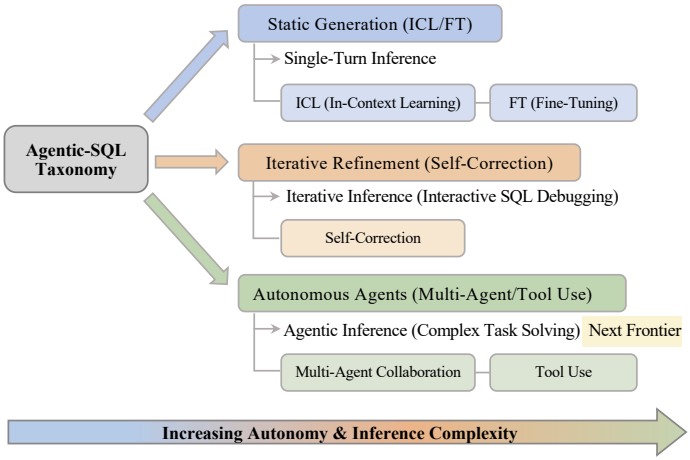

Fig. 2: The Agentic-SQL Taxonomy: Categorizing methods by inference autonomy from single-turn generation to multi-agent reasoning.

3.1 The Agentic-SQL Taxonomy

To better understand the current landscape, we propose an autonomy-based classification system as illustrated in Figure 2. We categorize methods based on their inference complexity rather than just their training data.

1. **Single-Turn Generation (Level 1):** The model receives the schema and question and produces a SQL query in a single pass. This is the simplest form of translation and is the baseline for most evaluations.
2. **Iterative Refinement (Level 2):** The system incorporates a feedback loop. If the initial SQL fails to execute or produces an error, the model receives the error message and attempts to fix the query. This level introduces basic self-correction.
3. **Agentic Collaboration (Level 3):** The task is distributed among multiple specialized agents. One agent might focus on schema filtering, another on query decomposition, and a third on final debugging. This level represents the state of the art in handling complex, real-world databases.

3.2 Formalizing In-Context Learning

In the ICL paradigm, we do not modify the model parameters θ. Instead, we construct a prompt P that guides the model π to generate the desired SQL $\hat{Y}$.

The prompt typically consists of an instruction I, a schema description S, and the user question Q. For zero-shot settings, the prompt is defined as:

$$P_0 = I \oplus S \oplus Q \tag{3}$$

For few-shot settings, we prepend k demonstration examples $E = \{e_1, e_2, \ldots, e_k\}$ to the prompt. Each example e_i consists of a natural language question and its corresponding gold SQL query. The few-shot prompt becomes:

$$P_k = E \oplus I \oplus S \oplus Q \tag{4}$$

The model generates the sequence by maximizing the conditional probability:

$$\hat{Y} = \text{argmax}_Y \prod_{t=1}^{L} P(y_t | y_{<t}, P_k; \theta) \tag{5}$$

While ICL is efficient because it avoids the high cost of training, the performance is highly sensitive to the selection of examples in E and the way the schema S is serialized into text.

3.3 Vanilla and Few-Shot Prompting

The simplest ICL approach is vanilla prompting, where the schema is presented as a list of CREATE TABLE statements [7]. However, research has shown that the order and relevance of few-shot examples significantly impact accuracy [23]. DAIL-SQL [13] addresses this by optimizing how examples are selected and formatted. DAIL-SQL uses a selection strategy based on the similarity between the target question and the candidate examples. Given a question Q, it calculates a similarity score $sim(Q, Q_i)$ for all questions in a training bank. The top k examples are selected to provide the model with the most relevant structural patterns. Furthermore, it employs a concise serialization format that reduces token count while preserving the relational structure. This focus on prompt efficiency is a key theme in Level 1 systems.

3.4 Decomposition-Based Reasoning

As questions become more complex, single-turn generation often fails. Level 2 systems use decomposition to break the problem into manageable steps. DIN-SQL [17] is a representative framework that implements a multi-stage reasoning pipeline. The DIN-SQL process begins with schema linking to identify the relevant tables and columns. After linking, the system performs query classification to determine the difficulty level. The probability of generating a correct query $\hat{Y}$ is decomposed into several intermediate steps:

$$P(\hat{Y}|P) = P(L|P) \cdot P(D|L, P) \cdot P(\hat{Y}|D, L, P) \tag{6}$$

In this formulation, L represents the schema links and D represents the decomposed sub-queries. By isolating these components, the model can focus

its attention on specific logic, such as join conditions or nested aggregations, without being overwhelmed by the entire schema.

4 Paradigm II: Fine-Tuning and Model Adaptation

While in-context learning offers flexibility, fine-tuning enables deeper specialization by adapting model weights to SQL syntax and structural constraints [11, 25]. This paradigm is especially valuable for open-source models, which can be optimized for specific domains or SQL dialects. In our taxonomy, fine-tuning marks the shift from general reasoning to domain-specific expertise.

4.1 Parameter-Efficient Fine-Tuning (PEFT)

Training billions of parameters for every new database task is computationally expensive. Parameter-Efficient Fine-Tuning (PEFT) has emerged as a solution that modifies only a small fraction of the model weights while keeping the majority of the backbone frozen [4, 5, 26]. Low-Rank Adaptation (LoRA) [8] is the most prominent technique in this category.

LoRA represents the weight updates as the product of two low-rank matrices. For a pre-trained weight matrix $W_0 \in \mathbb{R}^{d\times k}$, the update ΔW is constrained by a low-rank decomposition:

$$W = W_0 + \Delta W = W_0 + BA \tag{7}$$

In this formulation, $B \in \mathbb{R}^{d\times r}$ and $A \in \mathbb{R}^{r\times k}$, where the rank r is much smaller than the hidden dimensions d and k. During the training process, only A and B receive gradient updates. The forward pass is defined as:

$$h = W_0 x + BAx \tag{8}$$

By employing PEFT, researchers can adapt large models to specific SQL dialects like PostgreSQL or BigQuery without the risk of catastrophic forgetting of the original language capabilities.

4.2 Data Augmentation Strategies

Fine-tuning requires large amounts of high-quality, labeled data, which is often difficult to obtain for specialized domains. Data augmentation solves this by using LLMs to generate synthetic training pairs [2]. A common approach involves taking an existing database schema and prompting a teacher model to generate a variety of natural language questions and their corresponding SQL statements [1].

We can formalize the augmentation process as a two-step generation. First, the system generates a synthetic question Q_{syn} from a given schema S and a target SQL structure Y_{target}:

$$Q_{syn} \sim P(Q|S, Y_{target}; \theta_{teacher}) \tag{9}$$

Then, to ensure quality, a second model (or the same teacher model) verifies that the generated question actually maps back to the target SQL. This back-translation consistency helps filter out noisy or incorrect examples. Methods that employ these strategies often see significant gains in robustness, especially when dealing with synonyms or linguistic variations in user questions. By expanding the diversity of the training set, these models become less sensitive to the exact phrasing of a query, which is a major weakness of vanilla ICL systems.

4.3 Task-Specific Adaptation and Loss Functions

Beyond standard cross-entropy loss, some adaptation strategies incorporate task-specific rewards to align the model with execution correctness. This often involves Reinforcement Learning from Feedback (RLF) [21]. The model is rewarded when the generated SQL $\hat{Y}$ executes successfully and produces the correct result set R.

The objective function in these cases includes an execution-based term:

$$\mathcal{L}_{total} = \mathcal{L}_{SFT} + \lambda \mathbb{E}_{\hat{Y} \sim \pi}[R(\hat{Y}, Y_{gold})] \tag{10}$$

Here, $R(\hat{Y}, Y_{gold})$ is a reward function that returns a high value if the execution results match and zero otherwise. The hyperparameter λ controls the influence of the execution reward. This approach encourages the model to prioritize logical correctness over syntactic similarity to a reference query. It is this focus on functional outcomes that connects fine-tuning paradigms to the higher-level agentic frameworks where execution feedback becomes an integral part of the inference process itself. By combining efficient weight updates with high-quality synthetic data, these adapted models provide the necessary precision for autonomous database interaction [19].

5 Paradigm III: Agentic and Interactive Frameworks

The highest level of our Agentic-SQL Taxonomy involves systems that treat SQL generation not as a static translation task, but as a dynamic reasoning process. These Level 3 systems move away from the "one-shot" philosophy that dominated early LLMs research. Instead, they adopt architectures where the model can interact with its own output, an execution environment, or other specialized agents. This shift is prompted by the observation that even the most powerful models fail on complex joins or nested queries when forced to provide an answer in a single turn.

5.1 Execution-Guided Refinement

Execution-guided refinement represents the first step toward true autonomy [16]. In this framework, the system does not assume that its first generated SQL query

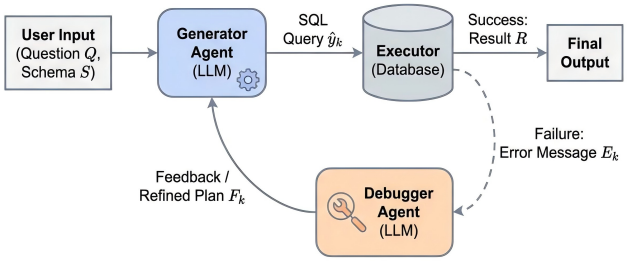

Fig. 3: The architecture of an agentic reasoning loop including planning, execution, and self-correction.

is correct. Instead, it executes the query against the target database and uses the resulting feedback to decide the next action. This feedback can take two forms: successful execution results or database engine error messages.

If the SQL execution fails, the system captures the error log (e.g., syntax errors, undefined columns, or join mismatches) and feeds it back into the model. We can formalize this iterative process. Let Y_t be the SQL query generated at iteration t. The execution function $E(Y_t, S)$ returns either a result set R or an error message ϵ. If an error occurs, the next query is generated as:

$$Y_{t+1} = \pi(I, Q, S, Y_t, \epsilon|\theta) \tag{11}$$

This loop continues until a valid result is produced or a maximum number of iterations is reached. DIN-SQL [17] effectively uses a self-correction module where the model is specifically prompted to "look at the error and the query" to find the bug. This mirrors the behavior of a human programmer who rarely writes perfect code on the first attempt. By allowing the model to see its own mistakes, these systems significantly reduce the number of "unexecutable" queries, a common failure mode for models like ChatGPT which can have incorrect execution rates of around 40% on challenging benchmarks.

5.2 Multi-Agent Collaboration

While iterative refinement focuses on a single model correcting itself, multi-agent collaboration (MAC) distributes the cognitive load across multiple specialized entities [6]. In these systems, each agent is assigned a specific role, such as a Planner, a Coder, or a Reviewer. This division of labor allows each agent to focus on a smaller, more manageable part of the problem.

For example, a Planner agent might be responsible for analyzing the natural language question and identifying the necessary tables and join conditions. It produces a high-level plan P. The Coder agent then takes this plan and translates it into SQL syntax. Finally, a Reviewer agent checks the SQL for logical consistency against the schema. This interaction can be modeled as a sequence

of state transitions where the collective goal is to maximize the probability of a correct query:

$$P(Y|Q,S) \approx P(Y|A_{review}, A_{code}, A_{plan}, Q, S) \tag{12}$$

MAC-SQL [22] is a notable framework in this category. It uses a coordinator to manage the dialogue between these agents. Interestingly, this approach helps mitigate the "distraction" problem often seen in long context windows. When a single model tries to handle schema filtering, join logic, and formatting all at once, it may lose track of subtle constraints. In a multi-agent system, the Coder agent only sees the filtered schema provided by the Planner, which simplifies the generation task. This modularity also makes the system easier to debug, as developers can pinpoint exactly which agent failed.

5.3 Interactive SQL Debugging

Interactive debugging introduces an external observer into the loop. This observer can be a human user or an automated "Critic" model designed to find logical flaws that do not necessarily trigger execution errors. For instance, a query might be syntactically perfect but answer the wrong question (e.g., using a MIN() function instead of a MAX() function).

The BIRD-CRITIC benchmark [14] was designed specifically to evaluate this capability. It tasks models with identifying and fixing errors in pre-existing SQL queries. This is a much harder task than generation because it requires the model to understand the intent behind a potentially broken piece of code. The interaction in these systems often involves a multi-turn dialogue:

Turn 1: User asks Q
Turn 2: Agent returns Y_1 with a rationale
Turn 3: User/Critic flags a logical flaw
Turn 4: Agent returns a corrected Y_2

This interactive paradigm is essential for enterprise applications where the cost of an incorrect query is high. By incorporating a verification step, these systems move toward a collaborative model of database interaction.

5.4 Performance and Efficiency Trade-offs

The transition to agentic frameworks is not without cost. While Paradigm III systems generally achieve higher Execution Accuracy (EX) on datasets like BIRD [14], they do so at the expense of higher latency and increased token usage. Each turn in a self-correction loop or each exchange between agents consumes thousands of tokens and requires a full forward pass of the LLM.

For "Easy" queries, the overhead of a multi-agent system often outweighs the benefits [24], sometimes even introducing errors through over-thinking. Therefore, modern systems are beginning to adopt a "routing" strategy. They use a simple model for easy questions and only trigger the full agentic pipeline when

the question complexity exceeds a certain threshold. This hybrid approach seeks to balance the high accuracy of Level 3 autonomy with the practical requirements of real-time database interfaces.

The shift toward these agentic and interactive frameworks marks the end of the view that Text-to-SQL is just a translation problem. By treating the task as an iterative reasoning process, researchers are finally beginning to tackle the complexities of real-world, large-scale database systems that were previously unreachable for smaller, non-agentic models.

6 Challenges and Future Directions

While the shift toward agentic and interactive frameworks has improved the state of the art, several fundamental obstacles remain. These challenges limit the adoption of LLM-based Text-to-SQL systems in production environments, particularly regarding data security, operational costs, and technical robustness.

6.1 Privacy-Preserving Text-to-SQL

Data privacy remains a major barrier to enterprise adoption. Standard in-context learning requires sending database schemas, and sometimes sample rows, to third-party LLM providers. For many organizations, exposing internal schema structures or sensitive column names violates strict compliance policies [20]. This creates tension between model performance and data security. One promising direction is schema obfuscation. Rather than sending raw table names, a local system can map them to abstract identifiers before querying the LLM. However, this can reduce the model's ability to reason about semantic relationships between columns. Future work must balance enabling LLMs to infer database logic while hiding sensitive content. Specialized local models such as CodeS [13] offer another path, keeping both data and inference inside the company firewall.

6.2 Efficiency vs. Accuracy Trade-off

The Agentic-SQL Taxonomy introduced in this survey highlights a growing problem with inference costs. Level 3 systems, such as MAC-SQL [22], achieve high execution accuracy by employing multiple agents and iterative refinement loops. While these loops help fix errors, they increase the total number of tokens generated per question by an order of magnitude. This leads to high latency and significant API costs. Recent benchmarks like BIRD [14] show that complex queries are where agentic reasoning provides the most value. For simpler queries, a single-turn model is often sufficient. A clear future direction involves the creation of adaptive routing mechanisms. These systems would evaluate the difficulty of a user question and the complexity of the schema to decide whether a cheap, single-turn model can handle the task or if the more expensive agentic pipeline is required. Developing early-exit strategies for refinement loops could also prevent the system from "over-correcting" queries that are already nearly perfect.

6.3 Cross-Dialect Generalization

Most academic research focuses on standard SQLite or PostgreSQL syntax. However, real-world data stacks use a variety of dialects including BigQuery, Snowflake, and ClickHouse [18]. These dialects often have unique window functions, specific date-time handling, and different execution optimization strategies. As shown in the Spider 2.0 results, current models perform poorly when faced with enterprise-level SQL requirements, often failing to reach even 25% accuracy. Closing this gap requires more than just larger training sets. Models need to understand the underlying relational algebra and how it maps to different dialect-specific implementations. Future work should focus on multi-dialect pre-training and the use of adapters that can be swapped based on the target database engine [9]. This would enable a single base model to maintain high performance across diverse cloud data warehouses without needing full fine-tuning for every environment.

6.4 Handling Massive and Dynamic Schemas

Current methods typically assume that the relevant schema components can fit within the model's context window. In practice, enterprise databases can contain thousands of tables and tens of thousands of columns. Furthermore, these schemas are not static; they change as new data is added or as business logic evolves. Designing RAG-based (Retrieval-Augmented Generation) schema [12] selection modules is essential for scaling to these environments. These modules must be accurate enough to find the needle in the haystack, as missing a single foreign key relationship can lead to an unexecutable join. Future agentic systems will likely include dedicated "Explorer" agents that browse the database metadata and documentation to build a temporary, localized schema context for each specific query. This dynamic schema discovery will be a requirement for the next generation of autonomous database interfaces.

7 Conclusion

This survey traces the evolution of Text-to-SQL from single-turn translation models to agentic, multi-step reasoning systems. Through the Agentic-SQL taxonomy, we categorize methods by inference complexity and show that, despite improvements from iterative refinement and architectural specialization, performance on enterprise-level benchmarks such as BIRD and Spider 2.0 remains limited. While feedback-based and specialized models improve robustness, they increase latency and token cost. Future work should balance reasoning depth with efficiency and develop privacy-aware mechanisms for real-world deployment, moving toward collaborative, domain-adaptive SQL agents.

Bibliography

[1] Deng, N., Chen, Y., Zhang, Y.: Recent advances in text-to-sql: A survey of what we have and what we expect. In: Proceedings of the 29th International conference on computational linguistics. pp. 2166–2187 (2022)

[2] Ding, B., Qin, C., Zhao, R., Luo, T., Li, X., Chen, G., Xia, W., Hu, J., Tuan, L.A., Joty, S.: Data augmentation using llms: Data perspectives, learning paradigms and challenges. In: Findings of the Association for Computational Linguistics: ACL 2024. pp. 1679–1705 (2024)

[3] Dong, Q., Li, L., Dai, D., Zheng, C., Ma, J., Li, R., Xia, H., Xu, J., Wu, Z., Chang, B., et al.: A survey on in-context learning. In: Proceedings of the 2024 conference on empirical methods in natural language processing. pp. 1107–1128 (2024)

[4] Fu, Z., Yang, H., So, A.M.C., Lam, W., Bing, L., Collier, N.: On the effectiveness of parameter-efficient fine-tuning. In: Proceedings of the AAAI conference on artificial intelligence. vol. 37, pp. 12799–12807 (2023)

[5] Han, Z., Gao, C., Liu, J., Zhang, J., Zhang, S.Q.: Parameter-efficient fine-tuning for large models: A comprehensive survey. arXiv preprint arXiv:2403.14608 (2024)

[6] Hong, S., Zhuge, M., Chen, J., Zheng, X., Cheng, Y., Wang, J., Zhang, C., Wang, Z., Yau, S.K.S., Lin, Z., et al.: Metagpt: Meta programming for a multi-agent collaborative framework. In: The twelfth international conference on learning representations (2023)

[7] Hong, Z., Yuan, Z., Zhang, Q., Chen, H., Dong, J., Huang, F., Huang, X.: Next-generation database interfaces: A survey of llm-based text-to-sql. IEEE Transactions on Knowledge and Data Engineering (2025)

[8] Hu, E.J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., Chen, W., et al.: Lora: Low-rank adaptation of large language models. Iclr **1**(2), 3 (2022)

[9] Huang, Y., Guo, J., Mao, W., Gao, C., Han, P., Liu, C., Ling, Q.: Exploring the landscape of text-to-sql with large language models: Progresses, challenges and opportunities. arXiv preprint arXiv:2505.23838 (2025)

[10] Hwang, W., Yim, J., Park, S., Seo, M.: A comprehensive exploration on wikisql with table-aware word contextualization. arXiv preprint arXiv:1902.01069 (2019)

[11] Kanburoğlu, A.B., Tek, F.B.: Text-to-sql: A methodical review of challenges and models. Turkish Journal of Electrical Engineering and Computer Sciences **32**(3), 403–419 (2024)

[12] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.t., Rocktäschel, T., et al.: Retrieval-augmented generation for knowledge-intensive nlp tasks. Advances in neural information processing systems **33**, 9459–9474 (2020)

[13] Li, H., Zhang, J., Liu, H., Fan, J., Zhang, X., Zhu, J., Wei, R., Pan, H., Li, C., Chen, H.: Codes: Towards building open-source language models for

text-to-sql. Proceedings of the ACM on Management of Data **2**(3), 1–28 (2024)
[14] Li, J., Hui, B., Qu, G., Yang, J., Li, B., Li, B., Wang, B., Qin, B., Geng, R., Huo, N., et al.: Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sqls. Advances in Neural Information Processing Systems **36**, 42330–42357 (2023)
[15] Liu, A., Hu, X., Wen, L., Yu, P.S.: A comprehensive evaluation of chatgpt's zero-shot text-to-sql capability. arXiv preprint arXiv:2303.13547 (2023)
[16] Mao, W., Wang, R., Guo, J., Zeng, J., Gao, C., Han, P., Liu, C.: Enhancing text-to-sql parsing through question rewriting and execution-guided refinement. In: Findings of the Association for Computational Linguistics: ACL 2024. pp. 2009–2024 (2024)
[17] Pourreza, M., Rafiei, D.: Din-sql: Decomposed in-context learning of text-to-sql with self-correction. Advances in neural information processing systems **36**, 36339–36348 (2023)
[18] Pourreza, M., Sun, R., Li, H., Miculicich, L., Pfister, T., Arik, S.O.: Sql-gen: Bridging the dialect gap for text-to-sql via synthetic data and model merging. arXiv preprint arXiv:2408.12733 (2024)
[19] Shi, L., Tang, Z., Zhang, N., Zhang, X., Yang, Z.: A survey on employing large language models for text-to-sql tasks. ACM Computing Surveys **58**(2), 1–37 (2025)
[20] Stoisser, J.L., Martell, M.B., MÃĪrtens, K., Phillips, L., Town, S.M., Donovan-Maiye, R., Fauqueur, J.: Query, don't train: Privacy-preserving tabular prediction from ehr data via sql queries. arXiv preprint arXiv:2505.21801 (2025)
[21] Sun, S., Liu, R., Lyu, J., Yang, J.W., Zhang, L., Li, X.: A large language model-driven reward design framework via dynamic feedback for reinforcement learning. Knowledge-Based Systems **326**, 114065 (2025)
[22] Wang, B., Ren, C., Yang, J., Liang, X., Bai, J., Chai, L., Yan, Z., Zhang, Q.W., Yin, D., Sun, X., et al.: Mac-sql: A multi-agent collaborative framework for text-to-sql. In: Proceedings of the 31st International Conference on Computational Linguistics. pp. 540–557 (2025)
[23] Wang, Y., Yao, Q., Kwok, J.T., Ni, L.M.: Generalizing from a few examples: A survey on few-shot learning. ACM computing surveys (csur) **53**(3), 1–34 (2020)
[24] Yu, T., Zhang, R., Yang, K., Yasunaga, M., Wang, D., Li, Z., Ma, J., Li, I., Yao, Q., Roman, S., et al.: Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task. In: Proceedings of the 2018 conference on empirical methods in natural language processing. pp. 3911–3921 (2018)
[25] Zhang, B., Ye, Y., Du, G., Hu, X., Li, Z., Yang, S., Liu, C.H., Zhao, R., Li, Z., Mao, H.: Benchmarking the text-to-sql capability of large language models: A comprehensive evaluation. arXiv preprint arXiv:2403.02951 (2024)
[26] Zhang, D., Feng, T., Xue, L., Wang, Y., Dong, Y., Tang, J.: Parameter-efficient fine-tuning for foundation models. arXiv preprint arXiv:2501.13787 (2025)